

Practical - 5

AIM: Given an undirected graph with V vertices and Edges, check whether it contains any cycle or not

PROGRAM CODE :-

```
#include <stdio.h>
#include <stdbool.h>

#define MAX 1000

bool is_cycle_util(int v, bool visited[], int parent, int graph[][MAX], int V) {
    visited[v] = true;

    for (int i = 0; i < V; i++) {
        if (graph[v][i] == 1) {
            if (!visited[i]) {
                if (is_cycle_util(i, visited, v, graph, V)) {
                    return true;
                }
            } else if (parent != i) {
                return true;
            }
        }
    }

    return false;
}

bool is_cycle(int graph[][MAX], int V) {
    bool visited[V];

    for (int i = 0; i < V; i++) {
        visited[i] = false;
    }

    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            if (is_cycle_util(i, visited, -1, graph, V)) {
                return true;
            }
        }
    }

    return false;
}
```

```
}  
}  
  
return false;  
}  
  
int main() {  
    int V, E;  
  
    printf("Enter the number of vertices: "); scanf("%d",  
    &V);  
  
    printf("Enter the number of edges: "); scanf("%d",  
    &E);  
  
    int graph[MAX][MAX];  
  
    for (int i = 0; i < V; i++) {  
        for (int j = 0; j < V; j++) {  
            graph[i][j] = 0;  
        }  
    }  
  
    printf("Enter the edges (u v): \n"); for  
    (int i = 0; i < E; i++) {  
        int u, v;  
        scanf("%d %d", &u, &v);  
        graph[u][v] = 1;  
        graph[v][u] = 1;  
    }  
  
    if (is_cycle(graph, V)) { printf("Graph  
        contains cycle\n");  
    } else {  
        printf("Graph doesn't contain cycle\n");  
    }  
  
    return 0;  
}
```

OUTPUT :-

Enter the number of vertices: 4 Enter
the number of edges: 6 Enter the
edges (u v):

0 1

2 3

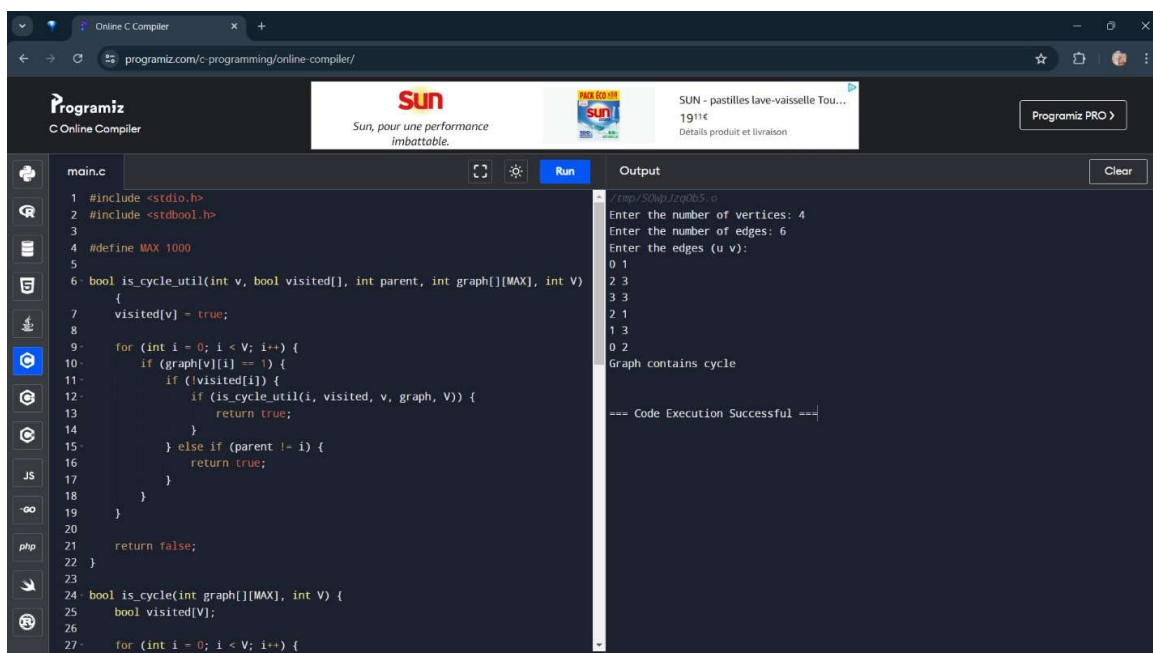
3 3

2 1

1 3

0 2

Graph contains cycle



```
main.c
1 #include <stdio.h>
2 #include <stdbool.h>
3
4 #define MAX 1000
5
6 bool is_cycle_util(int v, bool visited[], int parent, int graph[][MAX], int V)
7 {
8     visited[v] = true;
9     for (int i = 0; i < V; i++) {
10         if (graph[v][i] == 1) {
11             if (!visited[i]) {
12                 if (is_cycle_util(i, visited, v, graph, V)) {
13                     return true;
14                 }
15             } else if (parent != i) {
16                 return true;
17             }
18         }
19     }
20     return false;
21 }
22
23 bool is_cycle(int graph[][MAX], int V) {
24     bool visited[V];
25     for (int i = 0; i < V; i++) {
```

Output

```
Enter the number of vertices: 4
Enter the number of edges: 6
Enter the edges (u v):
0 1
2 3
3 3
2 1
1 3
0 2
Graph contains cycle
=== Code Execution Successful ===
```